Automação de Previsões Climáticas Copernicus

https://gemini.google.com/app/8010e216d84a059e

User prompt: Assuma o papel de um desenvolvedor especialista em meteorologia. O desafio será automatizar um processo de baixar vários modelos de previsão climática do copernicus. Com base nos diversos scripts python anexos, gere apenas 1, que precisará ter como dados de entrada, argparse, o modelo (originating_centre), o month e o year. O script deve ter uma biblioteca onde associa cada modelo a um system, conforme os scripts. Essa biblioteca deve estar preparada para receber outros modelos do Copernicus. Ao final do download, chega um zip com um nome aleatório, e dentro desse zip tem dois arquivos, data_1.nc e data_2.nc. O script deve descompactar e trocar o nome data_ por modelo_(0 e 1), a ser guardado no diretório saida_modelos. O segundo passo do script é chamar o python que faz as figuras usadas pelos meteorologistas operacionais. Esse script tem o nome gera_mapas_previsao.py, e a linha de comando para chamar o script é: python gera_mapas_previsao.py --f_previsao BOM_0.nc --f_hindcast BOM_1.nc --modelo BOM --mes 08 --ano 2026 Cada modelo acionado deve chamar o script com as opções corretas, inclusive apontando para o diretorio onde está o modelo. Opção extra e desejável: ter uma opção de acionamento único do script, que baixe um array de modelo inclusos na biblioteca, gerando todas as figuras de uma só vez. Atentar para opção do script rodar em crontab, portanto apontando corretamente os paths

Response: Aqui está o script em Python unificado para automação e execução via `crontab`. Ele integra a biblioteca de sistemas com base nas requisições do Copernicus (`seasonal-monthly-single-levels`), descompacta os arquivos `.nc`, organiza o diretório de saída e aciona a geração dos mapas operacionais.

```
#!/usr/bin/env python3
import argparse
import os
import subprocess
import sys
import zipfile
from pathlib import Path
import cdsapi

# Garante caminhos absolutos para compatibilidade com execução via CRONTAB
BASE_DIR = Path(__file__).resolve().parent
OUTPUT_DIR = BASE_DIR / "saida_modelos"

# Biblioteca de mapeamento: originating_centre -> system
MODEL_SYSTEM_MAP = {
    "bom": "2",
    "dwd": "22",
    "cmcc": "4",
    "ecmwf": "51",
    "meteo_france": "8",
    "ukmo": "60",
    "ncep": "2",
    "eccc": "3"
}

def baixar_e_processar_modelo(modelo: str, month: str, year: str) -> bool:
    """Baixa o arquivo do Copernicus CDS, extrai e renomeia para o padrão do gerador de mapas."""
    modelo_lower = modelo.lower()
    modelo_upper = modelo.upper()

    if modelo_lower not in MODEL_SYSTEM_MAP:
        print(f"[-] Erro: Modelo '{modelo}' não cadastrado na biblioteca.")
        return False

    system_id = MODEL_SYSTEM_MAP[modelo_lower]
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    # Formatação padronizada para mes/ano
    month_str = f"{int(month):02d}"
    year_str = str(year)

    temp_zip = BASE_DIR / f"temp_{modelo_lower}_{year_str}{month_str}.zip"

    request = {
        "originating_centre": modelo_lower,
        "system": system_id,
        "variable": [
            "maximum_2m_temperature_in_the_last_24_hours",
            "minimum_2m_temperature_in_the_last_24_hours",
            "total_precipitation"
        ],
        "product_type": [
            "ensemble_mean",
            "hindcast_climate_mean"
        ],
        "year": [year_str],
        "month": [month_str],
        "leadtime_month": ["1", "2", "3", "4", "5", "6"],
        "data_format": "netcdf",
        "area": [20, -110, -60, -20]
```

```
        }

        print(f"\n[+] Baixando dados do Copernicus para {modelo_upper} (Sistema {system_id})...")
        try:
            client = cdsapi.Client()
            client.retrieve("seasonal-monthly-single-levels", request, str(temp_zip))
        except Exception as e:
            print(f"[-] Erro ao realizar o download no Copernicus: {e}")
            return False

        # Processamento do arquivo ZIP baixado
        f_prev = OUTPUT_DIR / f"{modelo_upper}_0.nc"
        f_hind = OUTPUT_DIR / f"{modelo_upper}_1.nc"

        try:
            with zipfile.ZipFile(temp_zip, 'r') as zip_ref:
                for file_name in zip_ref.namelist():
                    if file_name == "data_1.nc":
                        zip_ref.extract(file_name, OUTPUT_DIR)
                        os.replace(OUTPUT_DIR / "data_1.nc", f_prev)
                    elif file_name == "data_2.nc":
                        zip_ref.extract(file_name, OUTPUT_DIR)
                        os.replace(OUTPUT_DIR / "data_2.nc", f_hind)
        except Exception as e:
            print(f"[-] Erro ao descompactar ou renomear arquivos: {e}")
            return False
        finally:
            if temp_zip.exists():
                temp_zip.unlink()

        print(f"[+] Arquivos extraídos: {f_prev.name} e {f_hind.name}")

        # Chamada do script de mapas com executable e caminhos absolutos
        script_mapas = BASE_DIR / "gera_mapas_previsao.py"
        cmd = [
            sys.executable, str(script_mapas),
            "--f_previsao", str(f_prev),
            "--f_hindcast", str(f_hind),
            "--modelo", modelo_upper,
            "--mes", month_str,
            "--ano", year_str
        ]

        print(f"[+] Gerando mapas: {' '.join(cmd)}")
        result = subprocess.run(cmd, capture_output=True, text=True)

        if result.returncode == 0:
            print(f"[+] Mapas gerados com sucesso para {modelo_upper}.")
            return True
        else:
            print(f"[-] Erro ao executar gera_mapas_previsao.py:\n{result.stderr}")
            return False

    def main():
        parser = argparse.ArgumentParser(description="Automação de download e geração de mapas operacionais do
        parser.add_argument("--modelo", type=str, choices=list(MODEL_SYSTEM_MAP.keys()) + ["ALL"], help="Orige
        parser.add_argument("--month", type=str, help="Mês da previsão (01 a 12).")
        parser.add_argument("--year", type=str, help="Ano da previsão (ex: 2026).")
        parser.add_argument("--all", action="store_true", help="Executa o download e a geração para todos os m

        args = parser.parse_args()

        if not args.month or not args.year:
            parser.error("Os argumentos --month e --year são obrigatórios.")

        if args.all or (args.modelo and args.modelo.upper() == "ALL"):
            modelos_para_rodar = list(MODEL_SYSTEM_MAP.keys())
        elif args.modelo:
            modelos_para_rodar = [args.modelo]
        else:
            parser.error("Informe um modelo com --modelo <NOME> ou utilize a opção --all.")

        for mod in modelos_para_rodar:
            baixar_e_processar_modelo(mod, args.month, args.year)

    if __name__ == "__main__":
        main()
```

Exemplo de Uso via Linha de Comando

- **Executar para um único modelo (Ex: ECMWF):**

```
python automacao_copernicus.py --modelo ecmwf --month 08 --year 2026
```

- **Executar para TODOS os modelos da biblioteca de uma vez:**

```
python automacao_copernicus.py --all --month 08 --year 2026
```

Exemplo de Configuração no Crontab

Ao agendar no `cron`, utilize sempre os caminhos absolutos do interpretador e do script para evitar erros de ambiente:

```
0 3 1 * * /usr/bin/python3 /caminho/absoluto/automacao_copernicus.py --all --month 08 --year 2026 >> /cami
```

User prompt: Analise o erro abaixo, bem como antes dele o log do download. Aparentemente o arquivo zip vem um ID ou nome aleatório. É preciso capturar esse nome, descompactar e então renomear e mover para o diretório final. python automacao_copernicus.py --modelo ecmwf --month 08 --year 2026 [+] Baixando dados do Copernicus para ECMWF (Sistema 51)... 2026-08-26 21:37:53,351 INFO Request ID is 51516ffa-5952-4196-a65c-dde75a12d184 2026-08-26 21:37:53,720 INFO status has been updated to accepted 2026-08-26 21:38:17,069 INFO status has been updated to running 2026-08-26 21:38:28,743 INFO status has been updated to successful [+] Arquivos extraídos: ECMWF_0.nc e ECMWF_1.nc
*[+] Gerando mapas: /home/marcio/miniconda3/bin/python /home/marcio/scripts/modelos/copernicus/gera_mapas_previsao.py --f_previsao /home/marcio/scripts/modelos/copernicus/saida_modelos/ECMWF_0.nc --f_hindcast /home/marcio/scripts/modelos/copernicus/saida_modelos/ECMWF_1.nc --modelo ECMWF --mes 08 --ano 2026 [-] Erro ao executar gera_mapas_previsao.py: Traceback (most recent call last): File "/home/marcio/miniconda3/lib/python3.13/site-packages/xarray/backends/file_manager.py", line 211, in _acquire_with_cache_info file = self._cache[self._key] ~~~~~~~~~~~~^^^^^^^^^^ File "/home/marcio/miniconda3/lib/python3.13/site-packages/xarray/backends/lru_cache.py", line 56, in __getitem__ value = self._cache[key] ~~~~~~~~~~~^^^^^ KeyError: [<class 'netCDF4._netCDF4.Dataset'>, ('/home/marcio/scripts/modelos/copernicus/saida_modelos/ECMWF_1.nc',), 'r', (('clobber', True), ('diskless', False), ('format', 'NETCDF4'), ('persist', False)), 'aabe3e56-4504-4b41-bbd4-4053abc8948b'] During handling of the above exception, another exception occurred: Traceback (most recent call last): File "/home/marcio/scripts/modelos/copernicus/gera_mapas_previsao.py", line 144, in <module> main() ~~~~^^ File "/home/marcio/scripts/modelos/copernicus/gera_mapas_previsao.py", line 110, in main ds_hind = xr.open_dataset(args.f_hindcast) File "/home/marcio/miniconda3/lib/python3.13/site-packages/xarray/backends/api.py", line 687, in open_dataset backend_ds = backend.open_dataset(filename_or_obj, ...<2 lines>... **kwargs,) File "/home/marcio/miniconda3/lib/python3.13/site-packages/xarray/backends/netCDF4_.py", line 666, in open_dataset store = NetCDF4DataStore.open(filename_or_obj, ...<8 lines>... autoclose=autoclose,) File "/home/marcio/miniconda3/lib/python3.13/site-packages/xarray/backends/netCDF4_.py", line 452, in open return cls(manager, group=group, mode=mode, lock=lock, autoclose=autoclose) File "/home/marcio/miniconda3/lib/python3.13/site-packages/xarray/backends/netCDF4_.py", line 393, in __init__ self.format = self.ds.data_model ^^^^^^^^ File "/home/marcio/miniconda3/lib/python3.13/site-packages/xarray/backends/netCDF4_.py", line 461, in ds return self._acquire() ~~~~~~~~~~~~~~^^ File "/home/marcio/miniconda3/lib/python3.13/site-packages/xarray/backends/netCDF4_.py", line 455, in _acquire with self._manager.acquire_context(needs_lock) as root: ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^ File "/home/marcio/miniconda3/lib/python3.13/contextlib.py", line 141, in __enter__ return next(self.gen) File "/home/marcio/miniconda3/lib/python3.13/site-packages/xarray/backends/file_manager.py", line 199, in acquire_context file, cached = self._acquire_with_cache_info(needs_lock) ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^ File "/home/marcio/miniconda3/lib/python3.13/site-packages/xarray/backends/file_manager.py", line 217, in _acquire_with_cache_info file = self._opener(*self._args, **kwargs) File "src/netCDF4/_netCDF4.pyx", line 2521, in netCDF4._netCDF4.Dataset.__init__ File "src/netCDF4/_netCDF4.pyx", line 2158, in netCDF4._netCDF4._ensure_nc_success FileNotFoundError: [Errno 2] No such file or directory: '/home/marcio/scripts/modelos/copernicus/saida_modelos/ECMWF_1.nc'*

Response: O erro `FileNotFoundError` ocorre porque o método `client.retrieve(...).download()` da biblioteca `cdsapi` baixa o arquivo no diretório atual com um nome aleatório ou padrão gerado pela API (geralmente uma string como `51516ffa-5952-4196-a65c-dde75a12d184.zip`), em vez de salvar diretamente no caminho de destino configurado em `temp_zip`.

Como o código anterior tentava abrir o arquivo `temp_zip` (que não chegou a ser criado com esse nome), a extração falhava silenciosamente e os arquivos `ECMWF_0.nc` e `ECMWF_1.nc` não eram gerados na pasta de destino, resultando na falha do `xarray` ao tentar abrir o arquivo inexistente.

Para resolver o problema, o método `.download()` do `cdsapi` deve receber explicitamente o caminho do arquivo de destino como argumento (`client.retrieve(...).download(target=...)`). O código abaixo corrige o download, insere uma validação para garantir a extração dos arquivos `.nc` dentro do `.zip` e faz o tratamento adequado dos caminhos.

```
#!/usr/bin/env python3
import argparse
import os
import subprocess
import sys
import zipfile
from pathlib import Path
import cdsapi

# Caminhos absolutos garantindo execução via CRONTAB
BASE_DIR = Path(__file__).resolve().parent
OUTPUT_DIR = BASE_DIR / "saida_modelos"

# Mapeamento: originating_centre -> system
MODEL_SYSTEM_MAP = {
    "bom": "2",
    "dwd": "22",
    "cmcc": "4",
    "ecmwf": "51",
    "meteo_france": "8",
```

```
        "ukmo": "60",
        "ncep": "2",
        "eccc": "3"
    }

    def baixar_e_processar_modelo(modelo: str, month: str, year: str) -> bool:
        """Baixa o ZIP do Copernicus, extrai data_1.nc e data_2.nc, renomeia e gera os mapas."""
        modelo_lower = modelo.lower()
        modelo_upper = modelo.upper()

        if modelo_lower not in MODEL_SYSTEM_MAP:
            print(f"[-] Erro: Modelo '{modelo}' não cadastrado na biblioteca.")
            return False

        system_id = MODEL_SYSTEM_MAP[modelo_lower]
        OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

        month_str = f"{int(month):02d}"
        year_str = str(year)

        # Definir caminho do arquivo zip temporário
        temp_zip = BASE_DIR / f"temp_{modelo_lower}_{year_str}{month_str}.zip"

        request = {
            "originating_centre": modelo_lower,
            "system": system_id,
            "variable": [
                "maximum_2m_temperature_in_the_last_24_hours",
                "minimum_2m_temperature_in_the_last_24_hours",
                "total_precipitation"
            ],
            "product_type": [
                "ensemble_mean",
                "hindcast_climate_mean"
            ],
            "year": [year_str],
            "month": [month_str],
            "leadtime_month": ["1", "2", "3", "4", "5", "6"],
            "data_format": "netcdf",
            "area": [20, -110, -60, -20]
        }

        print(f"\n[+] Baixando dados do Copernicus para {modelo_upper} (Sistema {system_id})...")
        try:
            client = cdsapi.Client()
            # Forçar o salvamento do download no caminho específico fornecendo 'target'
            result = client.retrieve("seasonal-monthly-single-levels", request)
            result.download(target=str(temp_zip))
        except Exception as e:
            print(f"[-] Erro ao realizar o download no Copernicus: {e}")
            return False

        f_prev = OUTPUT_DIR / f"{modelo_upper}_0.nc"
        f_hind = OUTPUT_DIR / f"{modelo_upper}_1.nc"

        # Processamento e descompactação do ZIP
        try:
            if not temp_zip.exists():
                print(f"[-] Erro: Arquivo descarregado não foi encontrado em {temp_zip}")
                return False

            with zipfile.ZipFile(temp_zip, 'r') as zip_ref:
                arquivos_no_zip = zip_ref.namelist()
                print(f"[+] Arquivos encontrados no ZIP: {arquivos_no_zip}")

                extracted_count = 0
                for file_name in arquivos_no_zip:
                    if file_name == "data_1.nc":
                        zip_ref.extract(file_name, OUTPUT_DIR)
                        os.replace(OUTPUT_DIR / "data_1.nc", f_prev)
                        extracted_count += 1
                    elif file_name == "data_2.nc":
                        zip_ref.extract(file_name, OUTPUT_DIR)
                        os.replace(OUTPUT_DIR / "data_2.nc", f_hind)
                        extracted_count += 1

                if extracted_count < 2:
                    print("[-] Aviso: Nem todos os arquivos esperados (data_1.nc, data_2.nc) estavam no ZIP.")
                    return False
```

```
        except Exception as e:
            print(f"[-] Erro ao descompactar ou renomear arquivos: {e}")
            return False
        finally:
            if temp_zip.exists():
                temp_zip.unlink()

        print(f"[+] Arquivos extraídos e salvos: {f_prev.name} e {f_hind.name}")

        # Execução do script de geração de mapas
        script_mapas = BASE_DIR / "gera_mapas_previsao.py"
        cmd = [
            sys.executable, str(script_mapas),
            "--f_previsao", str(f_prev),
            "--f_hindcast", str(f_hind),
            "--modelo", modelo_upper,
            "--mes", month_str,
            "--ano", year_str
        ]

        print(f"[+] Gerando mapas: {' '.join(cmd)}")
        res = subprocess.run(cmd, capture_output=True, text=True)

        if res.returncode == 0:
            print(f"[+] Mapas gerados com sucesso para {modelo_upper}.")
            if res.stdout:
                print(f"[Log gera_mapas]:\n{res.stdout}")
            return True
        else:
            print(f"[-] Erro ao executar gera_mapas_previsao.py:\n{res.stderr}")
            return False

    def main():
        parser = argparse.ArgumentParser(description="Automação de download e geração de mapas operacionais do
        parser.add_argument("--modelo", type=str, choices=list(MODEL_SYSTEM_MAP.keys()) + ["ALL", "all"], help
        parser.add_argument("--month", type=str, help="Mês da previsão (01 a 12).")
        parser.add_argument("--year", type=str, help="Ano da previsão (ex: 2026).")
        parser.add_argument("--all", action="store_true", help="Executa para todos os modelos cadastrados.")

        args = parser.parse_args()

        if not args.month or not args.year:
            parser.error("Os argumentos --month e --year são obrigatórios.")

        if args.all or (args.modelo and args.modelo.upper() == "ALL"):
            modelos_para_rodar = list(MODEL_SYSTEM_MAP.keys())
        elif args.modelo:
            modelos_para_rodar = [args.modelo]
        else:
            parser.error("Informe um modelo com --modelo <NOME> ou utilize a opção --all.")

        for mod in modelos_para_rodar:
            baixar_e_processar_modelo(mod, args.month, args.year)

    if __name__ == "__main__":
        main()
```
